

Test Quality Analysis

Feature 1 — Encrypted note persistence and user isolation (FR6-S / Sec 1, Story 5)

Test file: `server/app.test.ts` (108 lines, 2 tests) **Test plan:** `AstraNotes Planning/test_planning.md` (A1–A5 outlines)

Current test scope

- **Test 1:** Register user → create vault → encrypt note → PUT encrypted payload → GET list → decrypt and assert round-trip.
- **Test 2:** Register Carol → save note → register Dave → Dave PUT same note id → expect 403 → Carol's note still intact.

Identified weakness

Missing unauthenticated-access gate and validation rejection. The API has two obvious guard points with zero test coverage:

1. No request ever hits `/api/notes` without a session cookie. (Defect scenario: session middleware misconfig, regression, or a new route added without `requireAuth`.)
2. No request sends a malformed PUT body — e.g. `v: 3`, missing `ivB64`, non-JSON content type. The validator `isEncryptedNotePutBody` has no test coverage at any level.

These are the cheapest security regression tests to write: they exercise nothing but Express middleware and one pure function, yet they guard the app's core security boundary.

Improved test outline

Append to `server/app.test.ts`:

```
it('rejects unauthenticated GET, PUT, DELETE with 401', async () => {
  const r = request(app)
  await r.get('/api/notes').expect(401)
  await r.put('/api/notes/some-id').send({ v: 2, ivB64: 'x', ciphertextB64: 'y', updatedAt: 1 }).expect(401)
  await r.delete('/api/notes/some-id').expect(401)
})

it('rejects PUT with malformed encrypted payload (400)', async () => {
  // Register a user so auth passes, then send bad bodies
  const vault = new Vault()
  const meta = await vault.create('pw')
  const agent = request.agent(app)
  await agent.post('/api/register').send({ username: 'badput', password: 'pw', encryptionMeta: meta })

  // wrong version
  await agent.put('/api/notes/n1').send({ v: 1, ivB64: 'x', ciphertextB64: 'y', updatedAt: 1 }).expect(400)
  // missing ivB64
  await agent.put('/api/notes/n1').send({ v: 2, ciphertextB64: 'y', updatedAt: 1 }).expect(400)
  // missing ciphertextB64
  await agent.put('/api/notes/n1').send({ v: 2, ivB64: 'x', updatedAt: 1 }).expect(400)
  // body is plain string, not object
  await agent.put('/api/notes/n1').send('garbage').expect(400)
})

it('DELETE removes a note and rejects cross-user delete', async () => {
```

```

const v1 = new Vault(); const m1 = await v1.create('pw')
const a = request.agent(app)
await a.post('/api/register').send({ username: 'del1', password: 'pw', encryptionMeta: m1 }).expect(200)
const note = newNote({ title: 'delete me' })
const enc = await v1.encrypt(JSON.stringify(note))
await a.put(`/api/notes/${note.id}`).send({ v: 2, ivB64: enc.ivB64, ciphertextB64: enc.ciphertextB64,

// delete it
await a.delete(`/api/notes/${note.id}`).expect(204)
const list = await a.get('/api/notes').expect(200)
expect(list.body).toHaveLength(0)

// delete again - 404
await a.delete(`/api/notes/${note.id}`).expect(404)

// cross-user delete - 404 (the row is gone for everyone, but the endpoint scopes by user_id so it's
const v2 = new Vault(); const m2 = await v2.create('pw')
const b = request.agent(app)
await b.post('/api/register').send({ username: 'del2', password: 'pw', encryptionMeta: m2 }).expect(200)
await b.delete(`/api/notes/${note.id}`).expect(404)
})

```

What was wrong with the original

The original two tests cover the **happy encrypted round-trip** and the **cross-user PUT isolation**, which is good. But they skip entire HTTP method contracts (DELETE), security middleware (`requireAuth`), and request-body validation. A regression in any of those three areas would pass the existing suite silently.

Mocking assessment: helping or hiding risk?

No mocking is used — `openDb`, `createApp`, `Vault` (Web Crypto), `supertest` agent. This is **correct** for integration-level testing. Mocks would hide:

- Whether session middleware actually persists cookies across requests (`supertest` agent does this, and the test depends on it).
- Whether `saveEncryptedNoteForUser` runs the right SQL under the hood (the ownership check `SELECT user_id FROM notes WHERE id = ?` is exercised).
- Whether Web Crypto PBKDF2 derivation works in Node.js (not all `SubtleCrypto` implementations behave identically).
- Whether the validator rejects `v: 1` when the app expects `v: 2` (the `isEncryptedNotePutBody` check is tight to `NOTE_PAYLOAD_V2`).

The only thing missing is the DELETE endpoint at this level — but the test infrastructure to add it is already proven.

Coverage gap that still matters

1. **Silent corruption in `listNotesByUser`:** The repository has a `catch { /* skip corrupt row */ }` block (`server/repositories/noteRepository.ts:20-22`). If a payload column is not valid JSON, the note is silently dropped from the list. No test checks for this, and a corrupt DB could cause invisible data loss. A test that manually inserts a bad row and confirms it is skipped (and a warning logged) would be valuable — but the test cannot currently observe logger output.
2. **No concurrent-access test:** Two simultaneous PUTs to the same note id from the same user (last-write-wins) or from different users (ownership check) are untested. SQLite serializes writes, so a race shouldn't corrupt isolation, but the behavior is not locked down by any test.

3. **No session expiry / re-auth test:** What happens when a session expires mid-edit? The server returns 401, the client presumably handles it, but there's no test.
4. **PATCH /me/encryption-meta endpoint:** It exists in `authRouter.ts` but has zero test coverage.

Feature 2 — Tags (FR2b, Story 4)

Test file: `src/types/tags.test.ts` (17 lines, 3 tests) **Source:** `src/types/tags.ts` (24 lines)

Current test scope

Test	What it checks
normalizes case and dedupes	<code>['Work', 'work', ' Ideas ']</code> → <code>['work', 'ideas']</code>
caps tag count	37 tags → 32 tags (<code>MAX_TAGS_PER_NOTE</code>)
parses comma-separated input	<code>'a, B , a'</code> → <code>['a', 'b']</code>

Identified weakness

Tests under-specify the contract by a factor of ~5. The source code has four explicit behaviors with zero coverage:

Behavior in source	Covered?
Tag length truncation at <code>MAX_TAG_LENGTH</code> (40 chars)	No
Empty / whitespace tags silently dropped	Partial (whitespace in <code>' Ideas '</code> is trimmed, but an entirely-empty tag string is not tested standalone)
Semicolon and newline as separators in <code>parseTagsFromInput</code> (the regex is <code>/[,;\n]+/</code>)	No
Tag already at <code>MAX_TAGS_PER_NOTE</code> (boundary test: exactly 32)	No

These are not obscure edge cases — they are literal branches and regex tokens in the source. The tests say “yes” to 3 things but are silent on 4 others that a developer could break without knowing.

Improved test outline

Replace `src/types/tags.test.ts` with (or append):

```
import { describe, expect, it } from 'vitest'
import { MAX_TAGS_PER_NOTE, MAX_TAG_LENGTH, normalizeTags, parseTagsFromInput } from './tags'

describe('tags', () => {
  describe('normalizeTags', () => {
    it('normalizes case and dedupes', () => {
      expect(normalizeTags(['Work', 'work', ' Ideas '])).toEqual(['work', 'ideas'])
    })

    it('caps tag count at MAX_TAGS_PER_NOTE', () => {
```

```

    const many = Array.from({ length: MAX_TAGS_PER_NOTE + 5 }, (_, i) => `t${i}`)
    expect(normalizeTags(many)).toHaveLength(MAX_TAGS_PER_NOTE)
  })

  it('preserves order of first occurrence', () => {
    expect(normalizeTags(['z', 'y', 'z', 'x'])).toEqual(['z', 'y', 'x'])
  })

  it('drops empty and whitespace-only tags', () => {
    expect(normalizeTags(['', ' ', 'a', ''])).toEqual(['a'])
  })

  it('truncates tags longer than MAX_TAG_LENGTH', () => {
    const long = 'a'.repeat(MAX_TAG_LENGTH + 10)
    expect(normalizeTags([long])[0]).toHaveLength(MAX_TAG_LENGTH)
  })

  it('handles empty array', () => {
    expect(normalizeTags([])).toEqual([])
  })
})

describe('parseTagsFromInput', () => {
  it('parses comma-separated input', () => {
    expect(parseTagsFromInput('a, B , a')).toEqual(['a', 'b'])
  })

  it('parses semicolon-separated input', () => {
    expect(parseTagsFromInput('alpha; beta; Alpha')).toEqual(['alpha', 'beta'])
  })

  it('parses newline-separated input', () => {
    expect(parseTagsFromInput('x\ny\nx')).toEqual(['x', 'y'])
  })

  it('returns empty array for empty string', () => {
    expect(parseTagsFromInput('')).toEqual([])
  })

  it('returns empty array for whitespace-only string', () => {
    expect(parseTagsFromInput(' ')).toEqual([])
  })
})
})

```

That’s 11 tests instead of 3, covering every branch in `normalizeTags` and every regex alternative in `parseTagsFromInput`.

What was wrong with the original

The original tests only exercise the most obvious “happy path” behaviors. The 4 untested branches (length truncation, empty-tag dropping, semicolon/newline separators, empty input) are all simple regressions that a future refactor could introduce. For example, a well-intentioned change to `normalizeTags` that removes `.slice(0, MAX_TAG_LENGTH)` because “the UI already limits input” would silently break the backend contract — and no test would catch it.

The coverage gap is especially misleading because the test file exists and looks credible at a glance (3 tests, no obvious failures), but it does not cover the actual contract the rest of the app depends on.

Mocking assessment: helping or hiding risk?

No mocking is used — these are pure functions operating on strings and arrays. This is correct. The functions have zero side effects (no IO, no network, no random), so mocks would add noise. The remaining untested branches are also pure, so the fix is just more test cases, not test infrastructure.

Coverage gap that still matters

1. **Tag persistence in the integration layer:** Tags are stored inside the encrypted note JSON. The integration test (`server/app.test.ts`) asserts round-trip of the note title but never asserts that `tags` survive the encrypt → PUT → GET → decrypt pipeline. A bug that strips `tags` during encoding, or silently drops them on migration, would pass the existing tests. The fix is one assertion in the existing round-trip test: decrypt, parse, and check `note.tags`.
2. **Tag filter in `notesViewState`:** The view state test (`src/notes/notesViewState.test.ts`) checks tag filtering with exactly one tag (“school”). It does not test: no-tag matches, case mismatch in filter, partial matches, multiple tags intersection/union semantics. The current behavior is whatever the view-state code does — there is no explicit contract.
3. **Tag input UI component:** No component test exists for the tag editor (tag creation, deletion, paste handling, duplicate rejection, max-count enforcement). The `BlockPreview.test.tsx` is the only RTL component test in the entire client.

AI Assistance

The AI agent was particularly helpful at identifying the main weaknesses in the test structure. Although we are only looking at two features, its thinking showed that it found other issues and edge cases that could be patched as well.